

Problem Set 4 Lab

In this lab, you will incorporate Huffman Compression into the save portion of the Drawer program. For now, you should return a String version of the Huffman Table.

Files Changed

- `Explorer.java` - Added Load/Save buttons and use of Compression class
- `Compressor.java` - Scaffolding for Huffman Compression
- `CompressionTester.java` - Independent Tester for Huffman Compression
- `Drawer.java` - capture KeyEvents and forward to the active Workspace
- `Workspace.java`
 - add ability to handle KeyEvents and forward to active DrawingSpace

- add ability to translate to/from JSON
- `DrawingSpace.java`
 - handle ESC to deselect (Bonus Tier Expired)
 - handle BackSpace/Delete to delete shapes (Bonus Tier Expired)
 - add ability to translate to/from JSON
- `LineDrawlet.java`
 - Add ability to move with mouse (Bonus Tier Expired)
 - Add ability to move with keys (Bonus Tier Expired)

Setup

1. Download the base project and open it in IntelliJ.
2. Don't change the package name in this lab. If you do, you'll need to recreate the save file.
3. Read through the code and investigate the Explorer and Compressor classes. You will need to implement the compress method.
4. You will also need to implement a binary heap which can uses a Comparator similar to the BST from the last lab.

Implementation Details

- I have declared many the public methods and inner classes needed for this lab. You are not permitted to alter any public signatures without permission. You may add any private elements you need.
- The `compress` method will require many private method for each part of the compression:
 - `createFreqTable` - Create the frequency table from the input.
 - `createHuffmanTree` - Create the Huffman Tree from the frequency table. For the initial insertion, make sure to do so in lexicographical order!
 - `createHuffmanTable` - Create the Huffman Table from the tree.
- Do **not** implement all the above steps in one method.
- For the compression, you'll need to implement a Heap. This **must** be a separate class, and it needs to handle all comparable types.

- All non-private methods must have proper JavaDoc comments, and you must include comments with code that is not self-documenting.
- In the printed version of the Huffman Table, make sure all white space is readable. (That is, use "(enter)", "(space)", "(tab)", etc.)
- I have provided a Compressor tester for you to test the table separately if you'd like.
- You may want to test your heap separately from the program. I suggest using integers.

Rubric (Base Tier)

- Style/Documentation
 - Variable, Method, and Class names
 - JavaDoc comments on all non-private methods and classes
 - Proper Commenting -- including private methods
- Method Implementation
 - Heap class
 - constructor - two constructors to allow for a Comparator

- `add` - adds data to the heap
- `remove` - removes and returns the root of the heap
- `peek` - returns the root of the heap
- `createFrequencyTable` - return the frequency table
- `createHuffmanTree` - returns the root node of the Huffman Tree
- `createHuffmanTable` - return the table to be printed when the Save button is pressed

Additional Tiers (Complete 3 of 3 for a Late Day)

1. Add ability to draw text on an edge of the line
2. Add a custom tool (rectangle/circle) and provide a way to switch between tools
3. Add ability to add centered text within the rectangle/circle